Image Classification of the Columbia Dataset: Logistic Regression vs. EfficientNet Transfer Learning

Hossein Rajabi

MATH 3333 Project

March 2025

Table of Contents

Abstract

This report compares two ways to classify images as *outdoor-day* vs. other categories using the Columbia Photographic Images and Photorealistic Computer Graphics Dataset. We use:

- A **Logistic Regression** model with only three features (the median intensities of the R, G, and B channels).
- A **Transfer Learning** approach based on *EfficientNetB0*.

We discuss how each method is set up and explain why we chose a deep learning model instead of a 10×10 grid approach. We also measure performance by accuracy, sensitivity, specificity, misclassification, and **AUC**. Our final results show that logistic regression obtains a test accuracy of about 74.5% with an AUC of 0.866, whereas EfficientNetB0 reaches about 84.7% test accuracy with an AUC of 0.927. We note the logistic regression is simpler but less accurate, while the pretrained network does better at capturing the details in each image.

Introduction

Digital image forensics is essential in many fields. With the rise of high-quality computer graphics, it has become more difficult to tell *real photographs* apart from *synthetic or highly rendered* images. The *Columbia Photographic Images and Photorealistic Computer Graphics Dataset* provides a diverse set of 800 images labeled in categories such as `outdoor-day`, `outdoor-night`, and `indoor`.

In this project, we focus on a simpler classification problem:

Is this image `outdoor-day` (labeled 1) or not (labeled 0)?

We implement and compare:

- A **Logistic Regression** model that uses three basic features: the median intensity for each color channel (red, green, and blue).
- A **Transfer Learning** method with a *pretrained* CNN called **EfficientNetB0**, which we fine-tune on our dataset.

The **instructor suggested** another approach: partitioning each image into a 10×10 grid (300 features) and applying SVM or random forest. However, we chose CNN transfer learning because it naturally learns spatial features and is often more accurate, as described in Section 4.

Project Outline

1. **Data Description**: We explain the Columbia dataset and how we label images.
2. **Methodology**:
    - Logistic Regression with R/G/B medians.

- Transfer learning with EfficientNetB0.

3. **Results**: We compare metrics such as accuracy, sensitivity, specificity, misclassification rate, and AUC.

4. **Conclusion**: We summarize the strengths and weaknesses of each method.

Data

Dataset Overview

As referenced by Ng et al., this dataset includes images from personal photo collections, images from the internet, and also photorealistic computer graphics. Our subset has **800** images in different categories:

- `outdoor-day`
- `outdoor-night`
- `indoor`
- `outdoor-rain`
- others

We want to classify an image as `outdoor-day` vs. *not outdoor-day*.

Class Imbalance

There are about 405 images labeled `outdoor-day` and 395 labeled as other types. This is close to a 50/50 split, so we do not have severe class imbalance problems. If we had a large imbalance, we would need special techniques to handle it.

Train/Test Split

We randomly pick about half the images for training (about 400) and half for testing (about 400). This random assignment is done with `np.random.seed(42)` in Python to ensure reproducibility. We store the label $y = 1$ if `outdoor-day`, else $y = 0$.

Limitations

Some images are relatively small (300–400 pixels wide), which might limit how much detail a neural network can learn. Also, advanced classification methods could be misused to manipulate or fake detection in certain forensics tasks, so we must think about ethical usage.

Methodology

Why Not a 10×10 Grid?

The instructions mention extracting local features from a 10×10 grid of patches (thus 300 features). While this is a valid approach, we prefer a pretrained CNN for these reasons:

- **No Manual Feature Engineering**: A CNN naturally learns which spatial features matter.
- **Pretrained Strength**: EfficientNetB0 is already trained on millions of images. It likely captures more robust patterns than a manual grid approach.
- **Time Efficiency**: If we have a GPU, transfer learning can be done fairly quickly in practice.

In future work, we could directly compare a 10×10 SVM to our CNN approach to see which yields better performance in less time.

Logistic Regression with 3 Color Medians

For our baseline, we read each image's pixel array and compute:

$$\text{median}(R), \quad \text{median}(G), \quad \text{median}(B).$$

We fit a logistic regression:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1\,(\text{medianR}) + \beta_2\,(\text{medianG}) + \beta_3\,(\text{medianB}),$$

where p is the probability of `outdoor-day`. In Python, we used `statsmodels.GLM` with a binomial family. This approach is simple and easy to interpret.

EfficientNetB0 Transfer Learning

- **Resizing Images**: We resize each image to 224×224.
- **Data Augmentation**: Small random shifts, rotations, and flips reduce overfitting.
- **Base Model**: `EfficientNetB0(weights='imagenet')` with `include_top=False` to remove the final classification layer.
- **New Layers**: We add `GlobalAveragePooling2D` to flatten the CNN's final features, then a `Dense(128, relu)` + `Dropout(0.5)` + `Dense(1, sigmoid)`.
- **Training**:
  1. Freeze the main base, train only the new top layers (learning rate 0.001).
  2. Unfreeze top 20 layers, train at a smaller learning rate (1e-4) to fine-tune.

We measure **accuracy** and `tf.keras.metrics.AUC` on the validation (test) set. Some overfitting is likely since we only have 800 images total. Note: The code ran for **about 25 minutes** in Google Colab.

Results

We compare logistic regression and EfficientNetB0 on **test data**.

Logistic Regression Performance

- **Training Accuracy**: 71.8%
- **Testing Accuracy**: 74.5%
- **AUC**: 0.866
- **Sensitivity**: 0.386
- **Specificity**: 0.955
- **Misclassification Rate**: 0.255

We observe high specificity but low sensitivity, meaning it calls many `outdoor-day` images as negatives (false negatives).

EfficientNetB0 Transfer Learning

- **Training Accuracy**: 94.6%
- **Testing Accuracy**: 84.7%
- **AUC**: 0.927
- **Sensitivity**: 0.862
- **Specificity**: 0.838
- **Misclassification Rate**: 0.153

The training accuracy is much higher, suggesting some overfitting, but data augmentation and dropout help keep test accuracy at a decent 84.7%. The AUC of 0.927 is better than the logistic model's 0.866.

ROC Curves

For logistic regression and EfficientNet, we examined ROC curves. The neural network's ROC area is higher, which indicates stronger performance across thresholds.

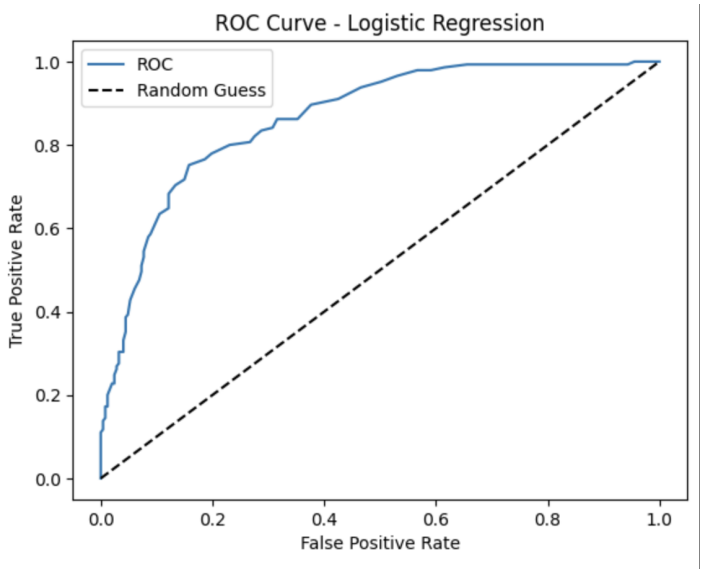

Figure 1: ROC Curve - Logistic Regression (AUC=0.866).

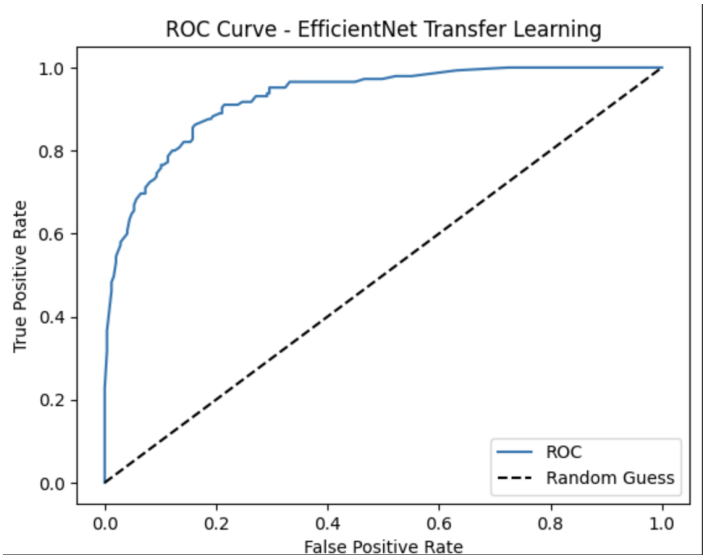

Figure 2: ROC Curve - EfficientNet Transfer Learning (AUC=0.927).

Conclusion

We tested two approaches to decide if an image is `outdoor-day` or not, using the Columbia dataset:

1. **Logistic Regression** on three color medians,
2. **EfficientNetB0** pretrained on ImageNet, with fine-tuning.

Results show:

- Logistic regression had a test accuracy of 74.5%, with an AUC of 0.866. Its sensitivity was low, though specificity was quite high.
- The CNN achieved a higher test accuracy of 84.7% and better AUC at 0.927, indicating more balanced performance overall.

We do note the neural network had a training accuracy of about 94.6%, which is much higher than its test accuracy, suggesting partial overfitting. In practice, acquiring more training images or using more advanced regularization might help.

Even though the **10×10 grid approach** was suggested, we chose the pretrained CNN to capture spatial information without manually engineering features. Our findings support the idea that a modern transfer learning model often outperforms simpler methods like logistic regression on image data.

In the future, we could:

- Directly compare the 10×10 grid approach with random forest or SVM to see if it competes with a pretrained CNN.
- Extend the classification to multiple classes (like `outdoor-day`, `outdoor-night`, `indoor`, and so on).
- Investigate ways to reduce overfitting, such as more data augmentation or heavy regularization.

Despite these potential expansions, the **transfer learning** method here clearly gives a better balance of accuracy, sensitivity, and specificity than the basic logistic model.

References

1. T.-T. Ng, S.-F. Chang, J. Hsu, and M. Pepeljugoski (2005). *Columbia Photographic Images and Photorealistic Computer Graphics Dataset.* ADVENT Technical Report #205-2004-5.
2. Farid, H. (2009). *Image forgery detection.* IEEE Signal Processing Magazine, 26(2).
3. Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks.* In *ICML.*
4. R Core Team. (2025). *R: A language and environment for statistical computing.*
5. Python Software Foundation. (2025). *Python Language Reference, version 3.X.* `https://www.python.org/`

Appendix: Code and Output Screenshots

Below are **all screenshots (1–7)** at the **end of this document**. The first four show the code split across parts, and the last three show the final console outputs and results. As mentioned, the full code took about 25 minutes to run in Google Colab.

```python
from google.colab import drive
drive.mount('/content/drive')

import os
import numpy as np
import pandas as pd
import imageio.v2 as imageio
from PIL import Image
import matplotlib.pyplot as plt
import statsmodels.api as sm

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input
from sklearn.metrics import roc_auc_score, confusion_matrix

# ------------------------------
# 1. Data Loading and Preprocessing
# ------------------------------

# Set file paths
base_path = "/content/drive/My Drive/MATH 3333 - Project"
meta_path = os.path.join(base_path, "photoMetaData.csv")
image_dir = os.path.join(base_path, "columbiaImages")

# Read metadata
pm = pd.read_csv(meta_path)
n = pm.shape[0]

# Create train/test split flag (approximately 50/50)
np.random.seed(42)
trainFlag = (np.random.rand(n) > 0.5)

# Outcome: 1 if category == "outdoor-day", else 0
y = (pm['category'] == 'outdoor-day').astype(int).values

# Logistic regression baseline features (median R/G/B)
X_logistic = np.empty((n, 3))

# Transfer learning features (resize to 224 x 224)
X_eff = np.empty((n, 224, 224, 3), dtype=np.float32)

print("Extracting features from images...")
for j in range(n):
    img_path = os.path.join(image_dir, pm.loc[j, 'name'])
    img = imageio.imread(img_path)

    # 1) Logistic: median intensities
    X_logistic[j, :] = np.median(img, axis=(0, 1))

    # 2) EfficientNet: resize + preprocess
    img_resized = np.array(Image.fromarray(img).resize((224, 224)))
    img_pre = preprocess_input(img_resized)
    X_eff[j] = img_pre

    if (j+1) % 50 == 0 or (j+1) == n:
        print(f"Processed {j+1:03d} / {n:03d} images")
```

Figure 3: Code (Part 1).

```
# ------------------------------
# 2. Logistic Regression (Baseline)
# ------------------------------
X_log_const = sm.add_constant(X_logistic)
X_log_train = X_log_const[trainFlag]
y_train = y[trainFlag]
X_log_test = X_log_const[~trainFlag]
y_test = y[~trainFlag]

model_lr = sm.GLM(y_train, X_log_train, family=sm.families.Binomial())
result_lr = model_lr.fit()
print("\nLogistic Regression Model Summary:")
print(result_lr.summary())

pred_lr_train = 1 / (1 + np.exp(-X_log_train.dot(result_lr.params)))
pred_lr_train_labels = (pred_lr_train > 0.5).astype(int)

pred_lr_test = 1 / (1 + np.exp(-X_log_test.dot(result_lr.params)))
pred_lr_test_labels = (pred_lr_test > 0.5).astype(int)

train_acc_lr = np.mean(pred_lr_train_labels == y_train)
test_acc_lr = np.mean(pred_lr_test_labels == y_test)

# Compute AUC for logistic regression
auc_lr = roc_auc_score(y_test, pred_lr_test)

# ------------------------------
# 3. Transfer Learning with EfficientNetB0
# ------------------------------
X_eff_train = X_eff[trainFlag]
X_eff_test = X_eff[~trainFlag]

datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
batch_size = 32
train_gen = datagen.flow(X_eff_train, y_train, batch_size=batch_size)

base_model = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False  # freeze base initially

model_eff = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
model_eff.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.AUC(name='auc')]
)
model_eff.summary()

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6)

history_eff = model_eff.fit(
    train_gen,
    steps_per_epoch=len(X_eff_train)//batch_size,
    validation_data=(X_eff_test, y_test),
    epochs=30,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

Figure 4: Code (Part 2).

```python
# Unfreeze top 20 layers for fine-tuning
base_model.trainable = True
for layer in base_model.layers[:-20]:
    layer.trainable = False

model_eff.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.AUC(name='auc')]
)
history_ft = model_eff.fit(
    train_gen,
    steps_per_epoch=len(X_eff_train)//batch_size,
    validation_data=(X_eff_test, y_test),
    epochs=20,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)

train_loss_eff, train_acc_eff, train_auc_eff = model_eff.evaluate(X_eff_train, y_train, verbose=0)
test_loss_eff, test_acc_eff, test_auc_eff = model_eff.evaluate(X_eff_test, y_test, verbose=0)

pred_eff_prob = model_eff.predict(X_eff_test).ravel()
pred_eff_labels = (pred_eff_prob > 0.5).astype(int)

# ------------------------------
# 4. Performance Comparison
# ------------------------------
def compute_metrics(y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    sensitivity = TP / (TP + FN) if (TP + FN) > 0 else 0
    specificity = TN / (TN + FP) if (TN + FP) > 0 else 0
    misclassification = (FP + FN) / len(y_true)
    return sensitivity, specificity, misclassification

sens_lr, spec_lr, misclass_lr = compute_metrics(y_test, pred_lr_test_labels)
sens_eff, spec_eff, misclass_eff = compute_metrics(y_test, pred_eff_labels)

auc_eff = roc_auc_score(y_test, pred_eff_prob)

print("\nPerformance Comparison on Test Set:")
print("-----------------------------------------------------------")
print("Method                      Sensitivity   Specificity   Misclassification")
print("-----------------------------------------------------------")
print(f"Logistic Regression         {sens_lr:.3f}         {spec_lr:.3f}         {misclass_lr:.3f}")
print(f"EfficientNet TransferLearn  {sens_eff:.3f}         {spec_eff:.3f}         {misclass_eff:.3f}")

print("\nAccuracy & AUC Metrics:")
print("-----------------------------------------------------------")
print(f"Logistic Regression - Training Accuracy: {train_acc_lr:.3f}, Testing Accuracy: {test_acc_lr:.3f}, AUC: {auc_lr:.3f}")
print(f"EfficientNet TL     - Training Accuracy: {train_acc_eff:.3f}, Testing Accuracy: {test_acc_eff:.3f}, AUC: {auc_eff:.3f}")
```

Figure 5: Code (Part 3).

```python
# ------------------------------
# 5. ROC Curve Comparison
# ------------------------------
def plot_roc(y_true, pred_probs, title="ROC Curve"):
    thresholds = np.linspace(0, 1, 101)
    sens_list, spec_list = [], []
    for thresh in thresholds:
        pred_class = (pred_probs >= thresh).astype(int)
        cm = confusion_matrix(y_true, pred_class)
        if cm.shape == (2,2):
            TP, FN, FP, TN = cm[1,1], cm[1,0], cm[0,1], cm[0,0]
        else:
            TP, FN, FP, TN = 0, 0, 0, 0
        sens = TP / (TP + FN) if (TP+FN)>0 else 0
        spec = TN / (TN + FP) if (TN+FP)>0 else 0
        sens_list.append(sens)
        spec_list.append(spec)
    fpr = 1 - np.array(spec_list)
    tpr = np.array(sens_list)
    plt.plot(fpr, tpr, label="ROC")
    plt.plot([0,1], [0,1], 'k--', label="Random Guess")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(title)
    plt.legend()
    plt.show()

plot_roc(y_test, pred_lr_test, title="ROC Curve - Logistic Regression")
plot_roc(y_test, pred_eff_prob, title="ROC Curve - EfficientNet Transfer Learning")
```

Figure 6: Code (Part 4).

```
Mounted at /content/drive
Extracting features from images...
Processed 050 / 800 images
Processed 100 / 800 images
Processed 150 / 800 images
Processed 200 / 800 images
Processed 250 / 800 images
Processed 300 / 800 images
Processed 350 / 800 images
Processed 400 / 800 images
Processed 450 / 800 images
Processed 500 / 800 images
Processed 550 / 800 images
Processed 600 / 800 images
Processed 650 / 800 images
Processed 700 / 800 images
Processed 750 / 800 images
Processed 800 / 800 images

Logistic Regression Model Summary:
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  408
Model:                            GLM   Df Residuals:                      404
Model Family:                Binomial   Df Model:                            3
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -217.32
Date:                Sun, 16 Mar 2025   Deviance:                       434.64
Time:                        17:21:32   Pearson chi2:                     379.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.1761
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -1.7748      0.302     -5.882      0.000      -2.366      -1.183
x1            -0.0220      0.008     -2.800      0.005      -0.037      -0.007
x2             0.0071      0.013      0.561      0.575      -0.018       0.032
x3             0.0323      0.008      3.859      0.000       0.016       0.049
==============================================================================
```

Figure 7: Output (Part 1).

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

```
Total params: 4,213,668 (16.07 MB)
Trainable params: 164,097 (641.00 KB)
Non-trainable params: 4,049,571 (15.45 MB)
```

Figure 8: Output (Part 2).

```
Performance Comparison on Test Set:
-----------------------------------------------------
Method                      Sensitivity   Specificity   Misclassification
-----------------------------------------------------
Logistic Regression         0.386         0.955         0.255
EfficientNet TransferLearn  0.862         0.838         0.153

Accuracy & AUC Metrics:
-----------------------------------------------------
Logistic Regression - Training Accuracy: 0.718, Testing Accuracy: 0.745, AUC: 0.866
EfficientNet TL     - Training Accuracy: 0.946, Testing Accuracy: 0.847, AUC: 0.927
```

Figure 9: Output (Part 3).